

which do you nerd out
the most about?

7 people are too cool :)

CS 340



Exam 1 Review



<https://forms.gle/H5KpayeBouPr6dySA>

Updates

80% + 20%



<https://forms.gle/H5KpayeBouPr6dySA>

1. Exam 1 - This Thursday
 - a. **Sign up now!**
 - b. **Study guide & practice exam out**
 - c. **No class Thursday**
2. HW4 - Due tomorrow night
3. MP 1 is graded **MP 1.2**
 - a. Let me know if you think there was a mistake
4. MP2 - Due Tuesday next week (Sept 29th)

PNG

Agenda

1. Review and Questions

2. HW 3 Coding Review

3. Practice Exam Coding Review

How I made the Exam...

T/F or conceptual → study guide

HW Questions → 10 points

Coding → 30 points

So total
10 points

How would I compile use_test.c and test.c?

↓ -uses test()
main()

↓
test()

clicker.cs.illinois.edu

Q2

~Code~
340



```
drschatz@cs-drschatz-MBP cs340 % cd /Users/drschatz/Documents/340-Sp26/cs340/term-demo
drschatz@cs-drschatz-MBP term-demo % ls
Makefile          sample2.py          test_c.dSYM
use_test_c.c
a.out              test_c               test_c.h
sample.py          test_c.c             test_python.py
drschatz@cs-drschatz-MBP term-demo % make clean
rm -f test_c test_c.o
drschatz@cs-drschatz-MBP term-demo % cd ../
drschatz@cs-drschatz-MBP cs340 %
```

gcc
term-demo/
test_c.c term-
demo/use_test_c.c

Follow up, which is true?

- ☒ A) The C code will compile on any computer that can compile C. *maybe*
- ☒ B) I can see the executable created in my file finder window after creating it in the terminal. *!!*
- ☒ C) The executable is in machine code, which would be the same on any computer. *-ISA*

clicker.cs.illinois.edu

Q3

~Code~
340



Big Ideas #1

ls, pwd, cd

compiling code creates an executable in your ISA
- That can run on your CPU

C code - portable

X don't share machine code
or provide versions

↓ fetch
decode
execute ↗

Pointer Math

int arr[3] = {1, 2, 3};

arr[1] == *(arr + 1)

pointer + size of (*pointer) * num

char *ptr; ← 0x1000

ptr = ptr + 2 ← 0x1002

int *ptr; ← 0x1000

ptr = ptr + 2; ← 0x1008

$$\frac{\text{arr} + 4}{1000} = 1004$$

int **ptr;

ptr = ptr + 1; ← 0x1008

(*ptr) ← int* ← 8 bytes

C-Strings

char *s = "by";

char s2[3] = "by";

function

strcpy

strcmp

→ copy by character

stack

□ s

b | y | \0

read only

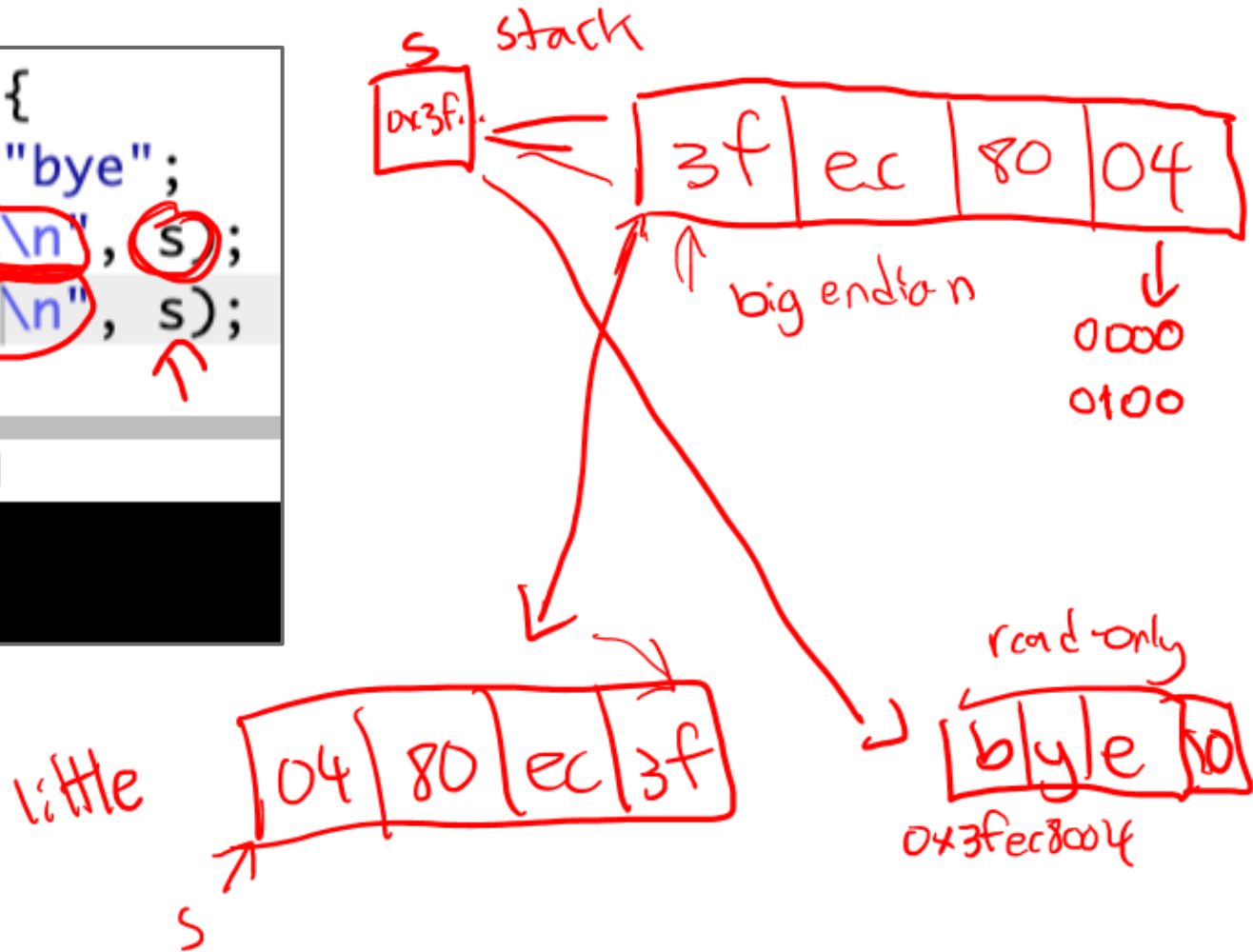
b | y | \0

```

11- int main(){
12- char *s = "bye";
13- printf("%x\n", s);
14- printf("%i\n", s);
15- }

```

3fec8004
1072463876



Where does s live?

```
11 int main(){  
12 char *s = "bye";  
13 printf("%x\n", s);  
14 printf("%i\n", s);  
15 }
```



3fec8004

1072463876

*s is
a char**

clicker.cs.illinois.edu

Q4

~Code~
340



- ☒ A) On the stack
- B) On the heap
- C) In read only memory

Where does *s live?

```
11 int main(){
12 char *s = "bye";
13 printf("%x\n", s);
14 printf("%i\n", s);
15 }
```

3fec8004
1072463876

clicker.cs.illinois.edu

Q5

~Code~
340



- A) On the stack
- B) On the heap
- C) In read only memory

**You computer uses little endian...
what do the bytes of s look like?**

```
11 int main(){  
12 char *s = "bye";  
13 printf("%x\n", s);  
14 printf("%i\n", s);  
15 }
```

ddd99004
-572944380

clicker.cs.illinois.edu

Q6

~Code~
340



- ~~A) dd d9 90 04~~
- ~~B) 57 29 44 38~~
- C) 04 90 d9 dd**
- ~~D) 38 44 29 57~~
- ~~E) 40 09 9d dd~~
- ~~F) 83 44 92 75~~

Why is everything stored as a 1 or 0 in a computer?

- A) The hardware is built to recognize and use 2 values of voltage.
- B) Binary is easier to work with then decimal.
- C) Caching only works with binary.
- D) Bytes are 1's and 0's.

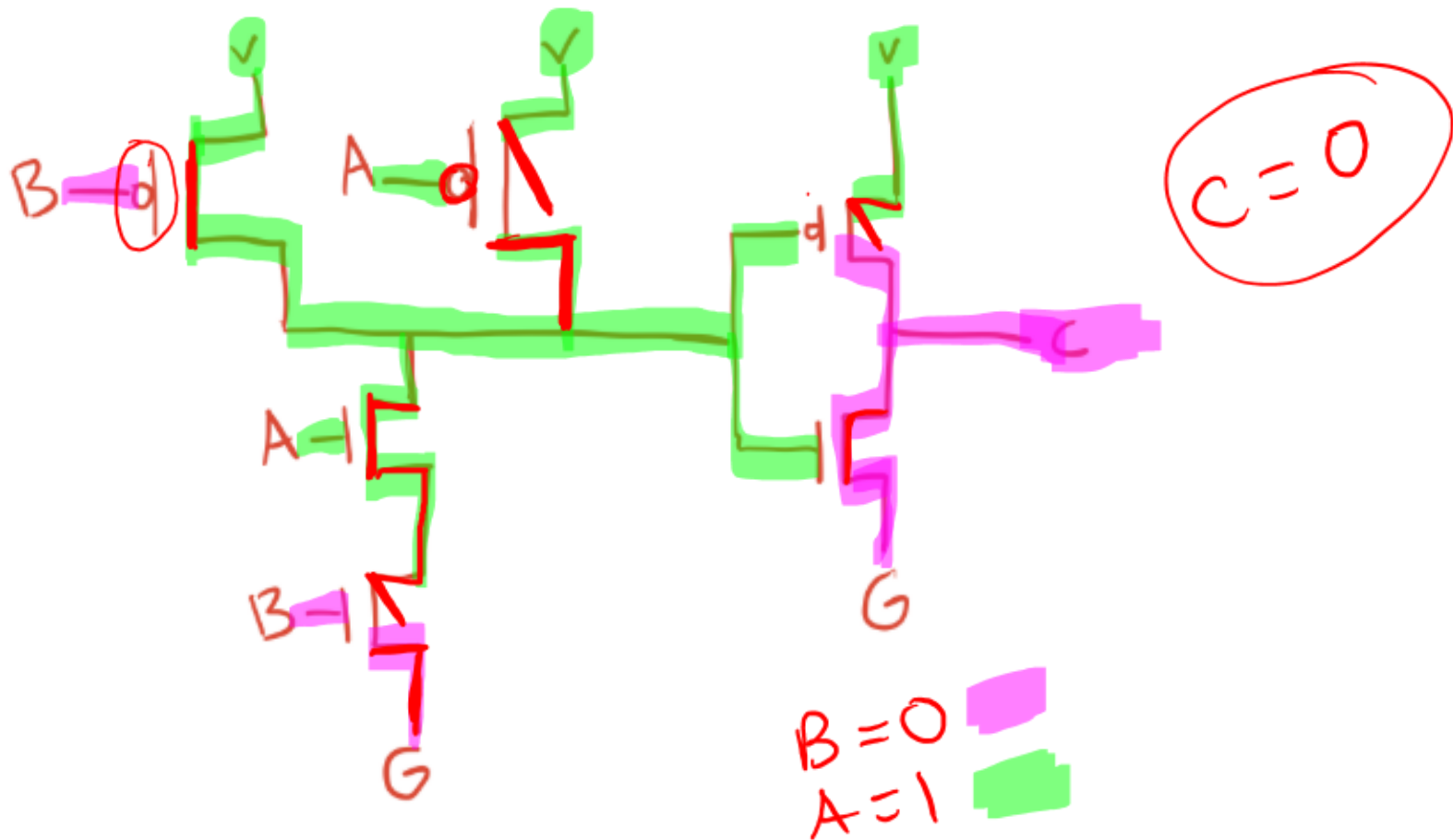


clicker.cs.illinois.edu

Q7

~Code~
340





What can we build with transistors?

- ~~A) Anything we can imagine~~ *Human level AI?*
- ☒ B) Any computational logic including selection and math *Halting problem*
- ~~C) Any computational logic including math but not selection~~
- ~~D) Only NAND NOT XOR logic~~

clicker.cs.illinois.edu

Q8

~Code~
340



What is the minimum depth of 2-MUX's needed for a 9 array selection?

$[1, 2, 3, 4, 5, 6, 7, 8, 9]$

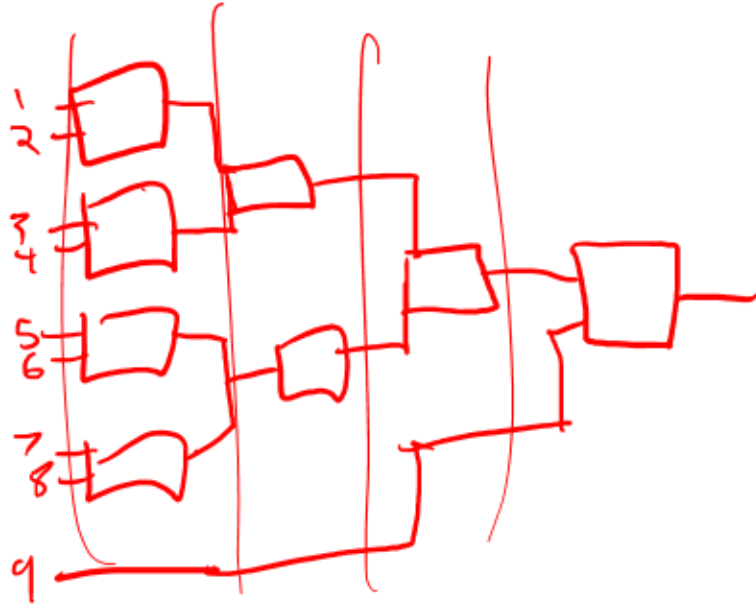
A) 1

B) 2

C) 3

D) 4

E) 5



clicker.cs.illinois.edu

Q9

~Code~
340



$\text{round up}(\log_2(9))$

Hardware for Storing Information

fast, expensive ← don't have
a lot
ex) ~~reg~~ d-flip-flop

RAM
SRAM

slow, Cheap ← a lot
SSD

caching to
utilize memory
types

What is true about caching?

- ~~A)~~ You cannot build a computer without caching.
- ☒ B) Your computer would be slower or bigger without caching
- ~~C)~~ The best caching algorithms puts a little bit of every program in the top layer of a cache for quick access.
- ~~D)~~ The CPU needs caching to work

clicker.cs.illinois.edu

Q10

~Code~
340



Caching - an algorithm for utilizing fast small memory and slow big memory.

Locality - the idea that computers often use nearby and similar information sequentially.

Spatial locality - *nearby*

Temporal locality - *reuse*

Temporal Locality Example from MP 1.

```
get_key(gd_GIF *gif, int key_size, uint8_t *sub_len, uint8_t *shift, uint8_t *byte)
{
    int bits_read;
    int rpad;
    int frag_size;
    uint16_t key;

    key = 0;
    for (bits_read = 0; bits_read < key_size; bits_read += frag_size) {
        rpad = (*shift + bits_read) % 8;
        if (rpad == 0) {
            /* Update byte. */
            if (*sub_len == 0) {
                read(gif->fd, sub_len, 1); /* Must be nonzero! */
                if (*sub_len == 0)
                    return 0x1000;
            }
            read(gif->fd, byte, 1);
            (*sub_len)--;
        }
        frag_size = MIN(key_size - bits_read, 8 - rpad);
        key |= ((uint16_t) ((*byte) >> rpad)) << bits_read;
    }
}
```

Decimal, Binary, Hexadecimal



F 3

0xF3

0x1F2

↓ ↓ ↓

0001 1111 0010

256 128 64 32 16 8 4 2

= 498

2³B

- 128

415

- 64

4511

- 32

19

- 16

3

Does the decimal value 498 fit in a char? What about an int?

$$0x1F2 = 498$$

3 digits



1-byte = 2 Hex digits

- A) Just a char, not an int
- ☒ B) Just an int, not a char
- C) Both

Coding Question on the Exam

1. open workspace
2. read instructions
3. write out step
4. implement each step
5. compile and run/test your code